

MPCA Module 2 — 5 Mark Questions & Answers

Q1. Explain the three-address, two-address, one-address, and zero-address instruction formats with examples and a comparison table.

1. Instruction Format Types

1. **Three-Address Format:** Specifies Opcode, Destination, and two Source Operands.

* *Syntax:* Opcode Dest, Src1, Src2

* *Example:* ADD A, B, C $\rightarrow$ Memory[A] = Memory[B] + Memory[C]

2. **Two-Address Format:** Specifies Opcode, Destination (also acts as Src1), and Source Operand.

* *Syntax:* Opcode Dest, Src

* *Example:* ADD A, B $\rightarrow$ Memory[A] = Memory[A] + Memory[B]

3. **One-Address Format:** Specifies Opcode and one Operand. Accumulator (AC) is implicit destination & source.

* *Syntax:* Opcode Address

* *Example:* LOAD B followed by ADD C $\rightarrow$ AC = AC + Memory[C]

4. **Zero-Address Format:** Specifies Opcode only. Operates implicitly on Stack Top (TOS).

* *Syntax:* Opcode

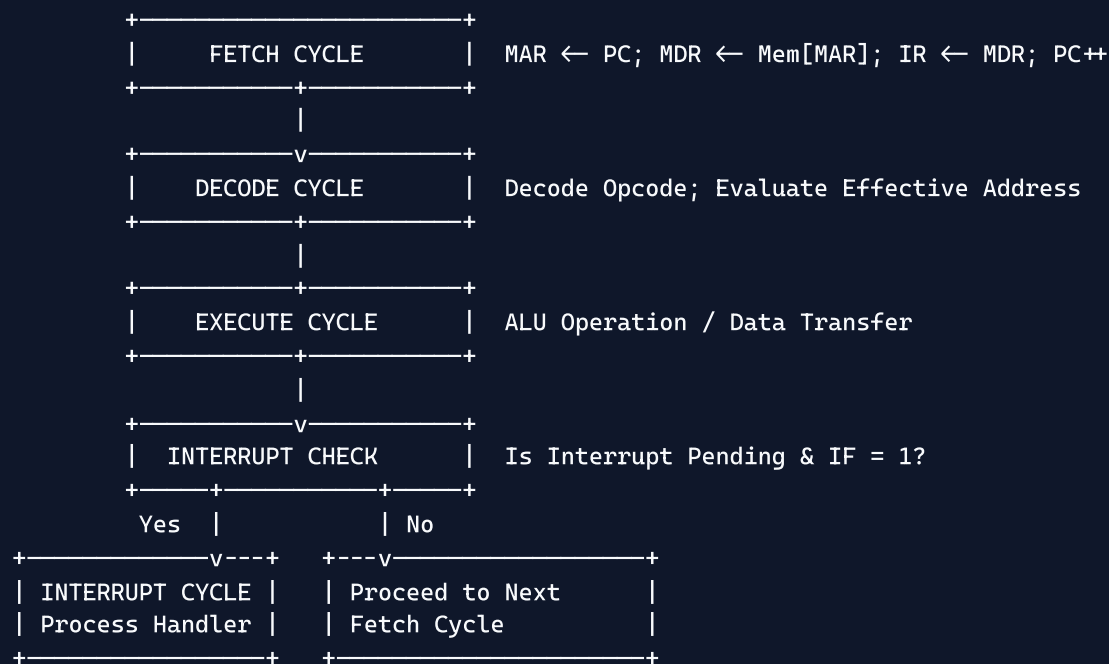
* *Example:* PUSH B , PUSH C , ADD $\rightarrow$ Pops B and C, pushes B + C onto stack.

2. Comparison Table

Format	Explicit Operands	Implicit Location	Instruction Length	Program Length (No. of Insts)
3-Address	3	None	Longest	Shortest
2-Address	2	None	Medium	Medium
1-Address	1	Accumulator (AC)	Short	Long
0-Address	0	Stack (TOS)	Shortest	Longest

Q2. Explain the basic instruction cycle with interrupt processing, including the interrupt processing cycle steps.

1. Combined Flowchart / Lifecycle



2. Interrupt Processing Cycle Steps

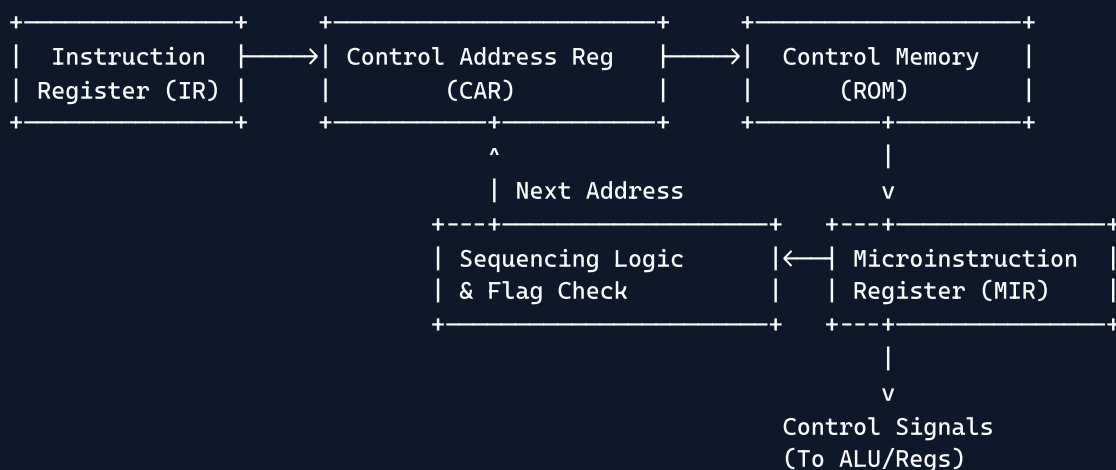
When an interrupt is detected at the end of the execute cycle:

1. **Push Status:** Push Flag Register, **CS**, and **IP** onto stack to save current CPU state.
2. **Disable Interrupts:** Clear Interrupt Flag (**IF = 0**).
3. **Fetch Vector Address:** Read Interrupt Vector from IVT based on interrupt type.
4. **Jump to ISR:** Load **CS** and **IP** with Interrupt Service Routine entry address.
5. **Resume (IRET):** At the end of ISR, **IRET** pops **IP**, **CS**, and Flags back to resume main execution seamlessly.

Q3. Explain the working of a microprogrammed (soft-wired) control unit with a block diagram and example (ADD AX, BX).

1. Concept & Block Diagram

A **Microprogrammed Control Unit** generates control signals by reading consecutive microinstructions from a specialized **Control Memory (ROM)**.



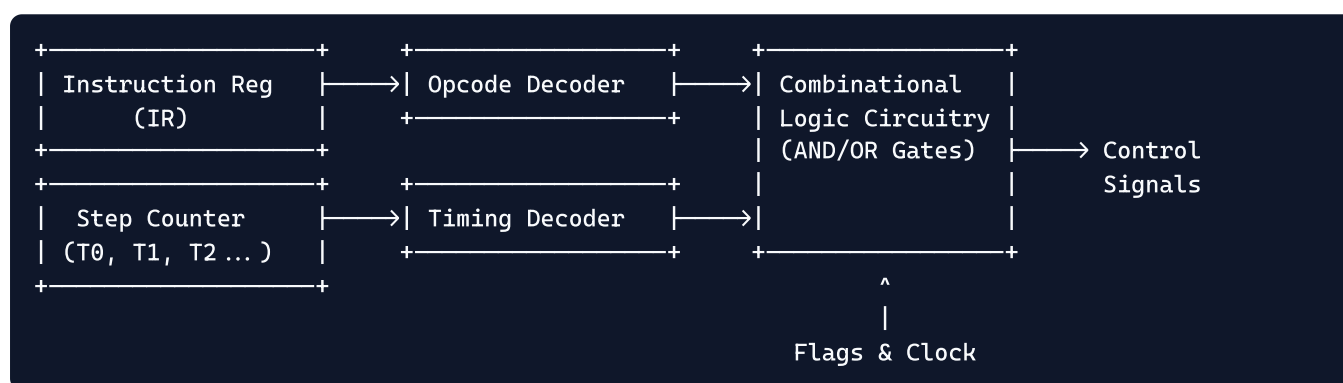
2. Working Step-by-Step for `ADD AX, BX`

1. Opcode of `ADD` in IR is mapped to Control Memory starting address (e.g., `Address 100`).
2. CAR is loaded with `Address 100` .
3. MIR reads microinstruction at `100` : issues control signals to transfer `AX` to ALU Temp Reg.
4. CAR increments to `101` : MIR issues control signals to transfer `BX` to ALU and trigger ALU `ADD` .
5. CAR increments to `102` : MIR issues control signals to write ALU result back into `AX` .

Q4. Explain the working of a hardwired control unit with a block diagram and example (`ADD AX, BX`).

1. Concept & Block Diagram

A **Hardwired Control Unit** uses fixed logic gates, decoders, and a step counter to generate control signals directly in real-time.



2. Working for `ADD AX, BX`

1. Opcode Decoder identifies `ADD` instruction.
2. Step Counter generates clock pulses `T0, T1, T2, T3` .
3. Combinational Logic derives signal equations:
 - * At `T2` : `Signal_ALU_Load = ADD · T2`
 - * At `T3` : `Signal_AX_Write = ADD · T3`
4. Operates at maximum hardware gate propagation speed.

Q5. Compare hardwired and microprogrammed control units in detail.

Comparison Parameter	Hardwired Control Unit	Microprogrammed Control Unit
Structure / Design	Combinational logic gates, decoders, counters.	Control Memory (ROM), CAR, MIR.
Execution Speed	Very Fast (direct gate delay).	Slower (ROM access latency).
Flexibility	Rigid; hardware modification required.	Flexible; update microprograms in ROM.
Cost	Higher initial design cost.	Lower development cost.
Instruction Complexity	Best for RISC (simple, fast).	Best for CISC (complex instructions).
Chip Area	Larger silicon footprint.	Smaller silicon footprint.

Q6. Explain microinstruction sequencing and execution with a suitable example (ADD AX, BX).

1. Sequencing & Execution Mechanism

Microinstruction sequencing determines the Control Memory address of the *next* microinstruction (Increment CAR, Branch on Flag, or Map Opcode).

Step Addr	Micro-Operation Control Signals	Next Address / Seq
100 (T0)	Temp1 ← AX	CAR ← CAR + 1 (101)
101 (T1)	Temp2 ← BX, ALU_Control = ADD	CAR ← CAR + 1 (102)
102 (T2)	AX ← ALU_Output	Map Next Opcode

- **Step 1 (100):** Transfers **AX** contents to temporary register. Next address = 101.
- **Step 2 (101):** Transmits **BX** to ALU, activates **ADD** operation. Next address = 102.
- **Step 3 (102):** Writes output to **AX** . Signals completion, loading CAR with next instruction opcode mapping.

Q7. Explain Flynn's classification of computer architecture with diagrams and real-life analogies.

SISD (Single Inst, Single Data) [CU] --→ [PU] --→ [Memory]	SIMD (Single Inst, Multiple Data) [CU] --+→ [PU1] --→ [Mem1] +→ [PU2] --→ [Mem2]
MISD (Multiple Inst, Single Data) [CU1] → [PU1] --+ [CU2] → [PU2] --+→ [Mem]	MIMD (Multiple Inst, Multiple Data) [CU1] → [PU1] ---→ [Mem1] [CU2] → [PU2] ---→ [Mem2]

1. **SISD:** Uniprocessor. Single instruction on single data.
* *Analogy:* Single teacher instructing 1 student. Example: 8086.
2. **SIMD:** Single instruction applied to multiple data items concurrently.
* *Analogy:* Drill sergeant giving 1 command to a squad of 50 soldiers. Example: GPUs, Vector processors.

3. **MISD:** Multiple instructions executed on a single data stream.
** Analogy:* Multiple doctors examining 1 patient. Example: Space Shuttle flight control redundant systems.
4. **MIMD:** Multiple independent processors executing different instructions on different data.
** Analogy:* Multiple workers doing different tasks independently in a factory. Example: Multi-core PCs, Clusters.

Q8. Explain instruction pipelining with a 5-stage pipeline timing diagram; calculate total cycles.

1. 5-Stage Pipeline Timing Diagram (4 Instructions: I1 to I4)

Clock Cycle	C1	C2	C3	C4	C5	C6	C7	C8
Inst 1 (I1)	IF	ID	EX	MEM	WB			
Inst 2 (I2)		IF	ID	EX	MEM	WB		
Inst 3 (I3)			IF	ID	EX	MEM	WB	
Inst 4 (I4)				IF	ID	EX	MEM	WB

2. Cycle Calculation for N = 4 instructions, K = 5 stages

- Non-Pipelined Cycles:** $N \times K = 4 \times 5 = 20 \text{ cycles}$.
- Pipelined Cycles:** $K + (N - 1) = 5 + (4 - 1) = 8 \text{ cycles}$.
- Speedup (\$\$):** $\frac{20}{8} = 2.5 \times \text{speedup}$.

Q9. Explain the three types of pipeline hazards (structural, data, control) with examples and their solutions.

- Structural Hazard:** Hardware resource conflict (e.g., IF and MEM stages accessing single RAM simultaneously).
** Solution:* Separate L1 Instruction & Data caches; or insert pipeline stall (bubble).
- Data Hazard:** Dependent instruction tries to read an operand before previous instruction writes it.
** Example:* ADD R1, R2, R3 followed by SUB R4, R1, R5 .
** Solution:* Operand Forwarding (bypassing), Stalling.
- Control Hazard:** Caused by branch instructions altering PC.
** Example:* JMP TARGET causes invalid prefetching.
** Solution:* Branch Prediction, Delayed Branching.

Q10. Explain pipeline hazard handling techniques: stalling, forwarding, branch prediction, and compiler scheduling.

- Stalling (Bubbles):** Inserts NOP (No Operation) cycles into the pipeline, delaying dependent stages until hazard resolves.
- Operand Forwarding (Bypassing):** Adds hardware data paths to send ALU result directly from EX/MEM output to the input of EX stage for the next instruction, eliminating stalls.
- Branch Prediction:** Predicts branch outcome (taken/not-taken) to keep fetching instructions without waiting for branch evaluation.

4. **Compiler Scheduling (Code Reordering):** Compiler reorders non-dependent instructions to fill hazard slots safely without changing program logic.
